Scraping Dynamic Pages with Selenium

"The web didn't stop at plain HTML and neither should your scraper."

Blog #02 · Web Scraping Series · Beginner Friendly Guide

[Pulkit Kashyap]

Learner · Builder · Blogger

March 8, 2026

selenium web-scraping python dynamic-pages beginner

So I was working on my first scraping project, pretty confident after getting BeautifulSoup to pull some data from a news site. Then I tried the same thing on a job listing site and got absolutely nothing back. No titles. No companies. No salaries. Just an empty page. I kept checking my code, thinking I'd made a mistake somewhere. Turns out, the page was loading everything through JavaScript after the initial HTML loaded. My scraper arrived before the content did. That's when I discovered ***Selenium****, and honestly, it changed everything.*

If you've ever run into this exact problem where your scraper returns nothing even though the page looks full of content in your browser this blog is for you. We're going to break it down in plain language, go step by step, and by the end you'll have a working Selenium scraper of your own.

Contents

1 First Things First What's a Dynamic Page? 3
1.1 A Real World Comparison 3

2 What is Selenium? 3
2.1 Selenium vs BeautifulSoup When to Use What 4

3 Setting Things Up 4
3.1 Step 1: Install Selenium 4

3.2 Step 2 Install a WebDriver 5
3.3 Step 3 Verify Your Setup 5

4 Understanding How Selenium Works **6**
4.1 Finding Elements on a Page 6

5 Waiting for Content The Most Important Part **7**
5.1 The Lazy Way `time.sleep()` 7
5.2 The Smart Way Explicit Waits 7

6 A Real Example Scraping a Dynamic Page **8**
6.1 Planning the Scraper 8
6.2 The Full Scraper 9

7 Interacting with Pages Beyond Just Extracting **11**
7.1 Clicking Buttons 11
7.2 Filling in Search Forms 12
7.3 Scrolling the Page 12

8 Handling Common Problems **13**
8.1 Element Not Found Error 13
8.2 Stale Element Error 13
8.3 Website Blocking Your Scraper 14

9 Ethical Reminders **14**

10 What I Wish Someone Had Told Me **15**

11 Wrapping Up **15**

First Things First What's a Dynamic Page?

Before we jump into Selenium, let's make sure we understand *why* we even need it.

When you open a website in your browser, one of two things is happening:

1. The server sends you a complete HTML page with all the content already baked in. This is a **static page**.
2. The server sends you a mostly empty HTML shell, and then JavaScript runs in your browser, fetches data from somewhere, and fills in the content. This is a **dynamic page**.

Tools like `requests` + `BeautifulSoup` only see what's in that initial HTML response step 1. They don't run JavaScript, so they never see the content added in step 2. That's the whole problem.

Quick Test

Right-click any webpage and click **"View Page Source"** (not Inspect). If you see your target data there it's static and BeautifulSoup is fine. If the source looks empty or shows placeholder divs only it's dynamic and you'll need Selenium.

A Real World Comparison

Think of it like ordering at a restaurant:

- **Static page** = The food is already on the table when you sit down.
- **Dynamic page** = You sit down, a waiter takes your order, and the kitchen prepares it. The table starts empty.

`requests` is like someone who only looks at the table when they first sit down. Selenium is someone patient enough to wait for the waiter to bring the food.

What is Selenium?

Selenium is a tool originally built for **automated browser testing** developers used it to test their websites by simulating real user actions (clicking buttons, filling forms,

scrolling). But the web scraping community quickly realized: *if it can simulate a real browser, it can also run JavaScript and load dynamic content!*

So Selenium basically opens a real browser (like Chrome or Firefox) in the background, loads the page just like you would, waits for JavaScript to finish, and *then* lets you extract the data. Brilliant, right?

Headless Mode

You don't have to watch a browser window pop up every time your scraper runs. Selenium supports **headless mode**, where the browser runs invisibly in the background. We'll use this in our code later.

Selenium vs BeautifulSoup When to Use What

	BeautifulSoup	Selenium
Best for	Static HTML pages	JavaScript-heavy pages
Speed	Very fast	Slower (real browser)
Complexity	Simple	Moderate
Can click buttons?	No	Yes
Can fill forms?	No	Yes
Can scroll?	No	Yes
Requires a browser?	No	Yes

The rule I follow: *start with BeautifulSoup, switch to Selenium only when you have to.* Selenium is powerful but slower, so don't use a sledgehammer where a regular hammer will do.

Setting Things Up

Okay, enough theory. Let's get our hands dirty. Here's everything you need installed before writing a single line of scraping code.

Step 1: Install Selenium

Open your terminal and run:

```
pip install selenium
```

Step 2 Install a WebDriver

Selenium needs a **WebDriver** a small program that acts as a bridge between your Python code and the actual browser. For Chrome, it's called **ChromeDriver**.

The easiest way today is to use `webdriver manager`, which automatically downloads the right driver for your browser version:

```
pip install webdriver-manager
```

Common Beginner Mistake

In older tutorials, you'll see people manually downloading ChromeDriver and pointing Selenium to a file path. This breaks every time Chrome updates. Use `webdriver-manager` instead it handles everything automatically. Saves you a lot of headaches!

Step 3 Verify Your Setup

Let's make sure everything works with a tiny test script:

Listing 1: Testing your Selenium setup

```python
from selenium import webdriver
from selenium.webdriver.chrome.service import Service
from selenium.webdriver.chrome.options import Options
from webdriver_manager.chrome import ChromeDriverManager

# Set up Chrome options
options = Options()
options.add_argument("--headless")    # Run in background (no window)

# Launch the browser
driver = webdriver.Chrome(
    service=Service(ChromeDriverManager().install()),
    options=options
)

# Open a page
driver.get("https://www.google.com")
print("Page title:", driver.title)
```

```

# Always close the browser when done!
driver.quit()
```

If you see `Page title: Google` in your terminal you're all set. Give yourself a little pat on the back, honestly.

Understanding How Selenium Works

Before jumping to a full example, let's understand the basic flow of any Selenium scraper you'll ever write. It's always the same four steps:

Step 1. **Launch** the browser (with your options)

Step 2. **Navigate** to the target URL

Step 3. **Wait** for the content to load

Step 4. **Extract** the data you need

Step 3 is the one most beginners skip, and it causes almost every Selenium bug you'll encounter. Always wait for content the browser is fast but JavaScript still takes a moment.

Finding Elements on a Page

Selenium finds elements using **locators**. Think of them as different ways of pointing at something on a page:

Listing 2: Common ways to find elements

```
from selenium.webdriver.common.by import By

# By CSS Selector (most flexible, recommended)
element = driver.find_element(By.CSS_SELECTOR, "div.product-title"
    )

# By ID (fastest when available)
element = driver.find_element(By.ID, "main-content")

# By class name
element = driver.find_element(By.CLASS_NAME, "price-tag")
```

```

# By XPath (powerful but harder to read)
element = driver.find_element(By.XPATH, "//h2[@class='title']")

# Find MULTIPLE elements (returns a list)
items = driver.find_elements(By.CSS_SELECTOR, "li.search-result")
```

How to Find CSS Selectors

Right-click any element on a webpage, click **Inspect**, then right-click the highlighted HTML in DevTools and choose **Copy** → **Copy selector**. Paste that directly into your code!

Waiting for Content The Most Important Part

I'm giving waits their own section because this is where 80% of Selenium bugs come from. Trust me on this.

There are two approaches: the lazy way and the smart way.

The Lazy Way `time.sleep()`

Listing 3: Simple but unreliable waiting

```
import time

driver.get("https://example.com/products")
time.sleep(3)    # Wait 3 seconds and hope for the best

products = driver.find_elements(By.CSS_SELECTOR, "div.product")
```

This works, but it's wasteful. If the page loads in 0.5 seconds, you've wasted 2.5 seconds. If the page takes 4 seconds one day, your scraper breaks. Not ideal.

The Smart Way Explicit Waits

Listing 4: Smart waiting with WebDriverWait

```
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
from selenium.webdriver.common.by import By
```

```python

driver.get("https://example.com/products")

# Wait UP TO 10 seconds for the element to appear
wait = WebDriverWait(driver, 10)
wait.until(
    EC.presence_of_element_located(
        (By.CSS_SELECTOR, "div.product-card")
    )
)

# Now safely extract
products = driver.find_elements(By.CSS_SELECTOR, "div.product-card")
print(f"Found {len(products)} products!")
```

This waits *exactly* as long as needed, up to a maximum of 10 seconds. Much cleaner. Use this in all your real projects.

Useful Expected Conditions

- `presence_of_element_located` element exists in DOM
- `visibility_of_element_located` element is visible
- `element_to_be_clickable` element can be clicked
- `text_to_be_present_in_element` element has text

A Real Example Scraping a Dynamic Page

Let's now put everything together in a realistic example. We'll build a scraper that collects book titles and prices from a test scraping site: books.toscrape.com a safe, legal practice website built specifically for learning scraping.

Planning the Scraper

Before writing code, always look at the page first:

1. Visit the site in your browser
2. Right-click a book title and hit **Inspect**

3. Note the HTML structure what tags and classes are used?
4. Find the **Next** button for pagination

On this site, each book is inside an `article.product_pod`, titles are in `h3 > a`, and prices are in `p.price_color`.

The Full Scraper

Listing 5: Full dynamic page scraper with pagination

```
from selenium import webdriver
from selenium.webdriver.chrome.service import Service
from selenium.webdriver.chrome.options import Options
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
from webdriver_manager.chrome import ChromeDriverManager
import pandas as pd
import time

#         Browser Setup


options = Options()
options.add_argument("--headless")
options.add_argument("--no-sandbox")
options.add_argument("--disable-dev-shm-usage")
options.add_argument(
    "user-agent=Mozilla/5.0 (Windows NT 10.0; Win64; x64) "
    "AppleWebKit/537.36"
)

driver = webdriver.Chrome(
    service=Service(ChromeDriverManager().install()),
    options=options
)
wait = WebDriverWait(driver, 10)

#         Scraping Logic


all_books = []
```

```python
url = "https://books.toscrape.com"

print("Starting scraper...")

for page_num in range(1, 4):   # Scrape first 3 pages
    print(f"  Scraping page {page_num}...")
    driver.get(url)

    # Wait for books to load
    wait.until(
        EC.presence_of_element_located(
            (By.CSS_SELECTOR, "article.product_pod")
        )
    )

    # Extract all books on this page
    books = driver.find_elements(
        By.CSS_SELECTOR, "article.product_pod"
    )

    for book in books:
        title = book.find_element(
            By.CSS_SELECTOR, "h3 a"
        ).get_attribute("title")

        price = book.find_element(
            By.CSS_SELECTOR, "p.price_color"
        ).text

        rating = book.find_element(
            By.CSS_SELECTOR, "p.star-rating"
        ).get_attribute("class").split()[-1]

        all_books.append({
            "title":  title,
            "price":  price,
            "rating": rating,
            "page":   page_num
        })

    # Click the Next button (if not on last page)
```

```
    try:
        next_btn = driver.find_element(
            By.CSS_SELECTOR, "li.next a"
        )
        url = next_btn.get_attribute("href")
        time.sleep(1)    # Small polite delay
    except:
        print("  No more pages found.")
        break

#         Save Results


driver.quit()

df = pd.DataFrame(all_books)
df.to_csv("books_scraped.csv", index=False)

print(f"\nDone! Scraped {len(df)} books.")
print(df.head())
```

Run this and you'll get a CSV file with book titles, prices, and ratings. Not bad for maybe 50 lines of code!

Interacting with Pages Beyond Just Extracting

One of Selenium's superpowers is that it can actually *interact* with a page, not just read it. This opens up a huge range of possibilities.

Clicking Buttons

Listing 6: Clicking elements on a page

```
# Click a "Load More" button
load_more = driver.find_element(By.ID, "load-more-btn")
load_more.click()

# Wait for new content to appear after click
wait.until(EC.presence_of_element_located(
```

```
    (By.CSS_SELECTOR, "div.new-items")
))
```

Filling in Search Forms

Listing 7: Typing into a search box

```
from selenium.webdriver.common.keys import Keys

# Find the search box, clear it, type, and hit Enter
search_box = driver.find_element(By.NAME, "q")
search_box.clear()
search_box.send_keys("python tutorials")
search_box.send_keys(Keys.RETURN)

# Wait for results to load
wait.until(EC.presence_of_element_located(
    (By.CSS_SELECTOR, "div.search-result")
))
```

Scrolling the Page

Some websites use **infinite scroll** content loads as you scroll down. Selenium can simulate this too:

Listing 8: Scrolling to load more content

```
import time

# Scroll down multiple times to load content
for i in range(5):
    driver.execute_script(
        "window.scrollTo(0, document.body.scrollHeight);"
    )
    time.sleep(2)    # Give the page time to load new content
    print(f"Scroll {i+1} done")

# Now extract all loaded content
items = driver.find_elements(By.CSS_SELECTOR, "div.post-card")
print(f"Found {len(items)} items after scrolling")
```

Handling Common Problems

Real-world scraping is messy. Here are the problems I've personally run into and how I fixed them.

Element Not Found Error

The most common error you'll see: `NoSuchElementException: no such element`

Listing 9: Safely finding elements that might not exist

```
# Instead of crashing, use a try-except
try:
    price = driver.find_element(
        By.CSS_SELECTOR, "span.discount-price"
    ).text
except:
    price = "N/A"    # Default if element doesn't exist
```

Stale Element Error

Sometimes the page refreshes after you've grabbed a reference to an element, making it "stale":

Listing 10: Re-finding elements after page changes

```
from selenium.common.exceptions import
   StaleElementReferenceException

for i in range(len(items)):
    try:
        # Re-find element fresh every time
        items = driver.find_elements(By.CSS_SELECTOR, "div.item")
        title = items[i].find_element(By.TAG_NAME, "h3").text
        print(title)
    except StaleElementReferenceException:
        print(f"Item {i} went stale, skipping...")
        continue
```

Website Blocking Your Scraper

Some sites detect and block automated browsers. A few tricks:

Listing 11: Making your scraper look more human

```
import time, random

options.add_argument("--disable-blink-features=
    AutomationControlled")
options.add_experimental_option(
    "excludeSwitches", ["enable-automation"]
)

# Add random delays between requests
def human_delay():
    time.sleep(random.uniform(1.5, 3.5))

# Use it between page loads
driver.get(url_1)
human_delay()
driver.get(url_2)
human_delay()
```

Ethical Reminders

I covered this in the first blog too, but it bears repeating here because Selenium is particularly powerful and can cause real harm to websites if misused.

- Always check **robots.txt** (e.g., `example.com/robots.txt`)
- Add delays between requests don't hammer a server
- Don't scrape personal or sensitive user data
- If a site has an API *use the API instead*
- Never use scraped data commercially without checking the ToS

A Note on Login-Gated Content

Selenium can technically log into websites and scrape content behind a login. However, scraping authenticated content almost always violates Terms of Service and can get your account banned or land you in legal trouble. Tread very carefully here.

What I Wish Someone Had Told Me

Looking back at when I started learning Selenium, here's the honest advice I'd give to my past self:

1. **Selenium is not always the answer.** A lot of dynamic sites actually load data via API calls in the background. Check the Network tab in DevTools you might be able to call that API directly with `requests`, which is 10x faster.
2. **DevTools is your best friend.** Learning to read the Network tab and Elements panel will make you a dramatically better scraper. Spend an hour just exploring it.
3. **Explicit waits always, `time.sleep` never** (or at least rarely). Your scrapers will be so much more reliable.
4. **Always call `driver.quit()`.** Forgetting this leaves zombie Chrome processes eating your RAM.
5. **Save raw HTML early.** When you're building a scraper, save the raw page HTML to a file first. Then test your parsing logic on that file, so you don't have to reload the page a hundred times.

Wrapping Up

Alright, let's recap what we covered in this blog:

- What dynamic pages are and why regular scrapers fail on them
- What Selenium is and when to actually use it
- How to set up Selenium from scratch (the modern way)

- Finding elements using CSS selectors and other locators
- Why waiting matters, and how to do it properly
- A complete real scraper with pagination
- Clicking, typing, and scrolling with Selenium
- Handling common errors gracefully
- Staying ethical and responsible

Selenium was one of those tools that felt overwhelming at first but clicked really fast once I started actually using it. The best thing you can do right now is go find a dynamic page you're curious about and try to scrape it. Break things, fix things, learn.

That's the whole game.

Thanks for reading! ❤

If this helped you, consider starring the repo and sharing it with someone who's learning too.

selenium dynamic-scraping python beginner-friendly web-scraping

Further Reading & Resources

- Selenium Official Docs: `https://selenium-python.readthedocs.io/`
- Practice Scraping Site: `https://books.toscrape.com`
- WebDriver Manager: `https://github.com/SergeyPirogov/webdriver_manager`
- Playwright (modern Selenium alternative): `https://playwright.dev/python/`
- Blog #01 in this series: *Web Scraping for Machine Learning*

Up Next in the Series

Blog #03: Building a Production Scraping Pipeline where we hook everything together with Scrapy, a database, and automated scheduling. Stay tuned!